

HastaneMVC — Teknik Referans Belgesi

ORTAM

Bileşen	Değer
Dil	C# 12
Çalışma zamanı	.NET 8
Framework	ASP.NET Core MVC
ORM	Entity Framework Core
Veritabanı	SQL Server LocalDB (<code>MSSQLLocalDB</code>) — <code>HastaneMVCDData</code>
Frontend	Bootstrap 5, Bootstrap Icons, jQuery
Bildirim	Toastr.js (CDN)
IDE	Visual Studio 2022
Sürüm kontrolü	Git, GitHub
Bağlantı dizesi	<code>"YOUR_CONNECTION_STRING"</code> (<code>appsettings.json</code> içinde <code>HastaneBaglantisi</code> adıyla)

PROJE YAPISI

```

HastaneMVC/
├── Controllers/
│   ├── BaseController.cs
│   ├── BransController.cs
│   ├── DoktorController.cs
│   ├── HastaController.cs
│   ├── HomeController.cs
│   ├── KayitliKisilerController.cs
│   ├── RandevuController.cs
│   ├── RandevuTahlilController.cs
│   └── TahlilTurController.cs
├── Models/
│   ├── BaseEntity.cs
│   ├── BransModel.cs
│   ├── DoktorModel.cs
│   ├── HastaModel.cs
│   ├── RandevuModel.cs
│   ├── RandevuTahlilModel.cs
│   ├── TahlilTurModel.cs
│   └── ErrorViewModel.cs
├── DTOs/
│   ├── BransModelDTO.cs
│   ├── DoktorModelDTO.cs
│   ├── HastaModelDTO.cs
│   ├── RandevuModelDTO.cs
│   ├── RandevuTahlilModelDTO.cs
│   └── TahlilTurModelDTO.cs
├── ViewModels/
│   ├── DoktorEkleVM.cs
│   ├── RandevuEkleVM.cs
│   ├── RandevuSilVM.cs
│   └── RandevuTahlilEkleVM.cs
├── Services/
│   ├── BaseService.cs
│   ├── DoktorService.cs
│   └── RandevuService.cs
├── Data/
│   └── HastaneContext.cs
├── Views/
│   ├── Shared/
│   │   ├── _Layout.cshtml
│   │   ├── _ValidationScriptsPartial.cshtml
│   │   ├── Toast.cshtml
│   │   └── Error.cshtml
│   ├── Brans/
│   │   ├── Index.cshtml
│   │   ├── Ekle.cshtml
│   │   ├── Guncelle.cshtml
│   │   └── Sil.cshtml
│   ├── Doktor/
│   │   ├── Index.cshtml
│   │   ├── Ekle.cshtml
│   │   ├── Guncelle.cshtml
│   │   └── Sil.cshtml
│   ├── Hasta/
│   │   ├── Index.cshtml
│   │   └── Ekle.cshtml

```

```

├── Guncelle.cshtml
├── Sil.cshtml
├── Randevu/
│   ├── Index.cshtml
│   ├── Ekle.cshtml
│   ├── Guncelle.cshtml
│   └── Sil.cshtml
├── RandevuTahlil/
│   ├── Index.cshtml
│   ├── Ekle.cshtml
│   ├── Guncelle.cshtml
│   └── Sil.cshtml
├── TahlilTur/
│   ├── Index.cshtml
│   ├── Ekle.cshtml
│   ├── Guncelle.cshtml
│   └── Sil.cshtml
├── Migrations/
├── wwwroot/
├── Program.cs
└── appsettings.json

```

İÇİNDEKİLER

Bölüm I — Dil ve Framework Kavramları

1. Koleksiyon Türleri: `List` , `IEnumerable` , `ICollection`
2. Entity Framework vs Entity Framework Core
3. Üç Katmanlı Mimari
4. LINQ
5. DTO ve ViewModel
6. Tag Helpers
7. AJAX ve jQuery
8. Model Validation
9. Toastr Bildirimleri
10. Soft Delete
11. Middleware
12. Base Sınıflar ve Kalıtım
13. Dependency Injection
14. .NET 8 Program.cs

Bölüm II — HastaneMVC Proje Özellikleri

1. Veri Modelleri ve Navigation Property
2. Branş Yönetimi (CRUD)
3. Doktor Yönetimi (CRUD)
4. Hasta Yönetimi (CRUD + Unique TC)

5. Randevu Yönetimi (CRUD + AJAX)
6. Tahlil Türü Yönetimi (CRUD)
7. Tahlil Sonuç Yönetimi (CRUD)

Ekler

- A. Connection String ve Migration Komutları
 - B. Git ile Kayıt ve GitHub'a Yükleme
-

BÖLÜM I — DİL VE FRAMEWORK KAVRAMLARI

1. Koleksiyon Türleri: `List`, `IEnumerable`, `ICollection`

1.1 Genel Bakış

C#’ta koleksiyonlar bir hiyerarşi içinde tanımlanır. En tepede `IEnumerable<T>` arayüzü bulunur ve yalnızca üzerinde döngü kurmaya izin verir. `ICollection<T>` bu yeteneğe öge ekleme, çıkarma ve sayma yeteneklerini ekler. `List<T>` ise indeksli erişim ve sıralama gibi tüm yetenekleri içeren somut sınıftır.

Hiyerarşi şu yöndedir: `IEnumerable<T>` → `ICollection<T>` → `IList<T>` → `List<T>`. Üst arayüzler daha az yeteneğe sahiptir ama daha geneldir.

1.2 Karşılaştırma Tablosu

Yetenek	<code>IEnumerable<T></code>	<code>ICollection<T></code>	<code>List<T></code>
<code>foreach</code> ile dolaşma	✓	✓	✓
<code>Add</code> , <code>Remove</code>	✗	✓	✓
<code>Count</code>	✗	✓	✓
İndeks ile erişim (<code>list[0]</code>)	✗	✗	✓
Sıralama (<code>Sort</code>)	✗	✗	✓
LINQ metotları	✓	✓	✓

1.3 Örnekler

Örnek 1.3.1 — Temel kullanım

```
// Konum: bilgilendirme amaçlı; HastaneContext dışında
IEnumerable<int> sayilar = new List<int> { 1, 2, 3 };
foreach (var s in sayilar)
{
    Console.WriteLine(s);
}
// sayilar.Add(4); // DERLEME HATASI - IEnumerable Add içermez
```

Çıktı:

1
2
3

Örnek 1.3.2 — Navigation property tipinde kullanım

```
// HastaneMVC/Models/TahlilTurModel.cs
public class TahlilTurModel : BaseEntity
{
    [Required(ErrorMessage = "Zorunlu alan")]
    [StringLength(150)]
    public string Ad { get; set; }

    public ICollection<RandevuTahlilModel> RandevuTahlilleri { get; set; }
}
```

Çıktı: (yok — yan etki: EF Core bu özelliği navigation property olarak yorumlar, fiziksel kolon üretmez.)

1.4 Mini Sözlük

- **Koleksiyon (collection):** Birden çok öğeyi bir arada tutan veri yapısı.
- **Arayüz (interface):** Bir sınıfın hangi metotları sağlamasını gerektiğini tanımlayan sözleşme.
- **Navigation property (gezinti özelliği):** EF Core'da iki entity arasındaki ilişkiyi C# tarafında temsil eden özellik.
- **Generic tip (<T>):** Tür parametresi alan, derleme anında somutlaşan veri yapısı.
- **foreach :** Bir koleksiyonun tüm öğelerini dolaşmak için kullanılan döngü yapısı.

1.5 Bölüm Özeti

- **IEnumerable<T>** yalnızca okuma için, **ICollection<T>** ekleme/silme için, **List<T>** indeksli erişim için.
- Navigation property'lerde **ICollection<T>** tercih edilir; EF Core ilişkiyi doğru kurar.
- LINQ metotları her üç tip üzerinde de çalışır.

2. Entity Framework vs Entity Framework Core

2.1 Genel Bakış

Entity Framework (EF), .NET Framework için geliştirilmiş klasik ORM kütüphanesidir ve yalnızca Windows üzerinde çalışır. Entity Framework Core (EF Core), .NET Core ile birlikte sıfırdan yazılmış, modüler, cross-platform halefidir. HastaneMVC projesi .NET 8 üzerinde EF Core kullanır.

2.2 Karşılaştırma Tablosu

Özellik	Entity Framework	Entity Framework Core
Platform	Sadece Windows (.NET Framework)	Cross-platform (.NET 6/7/8)
Performans	Daha yavaş	Daha hızlı
Mimari	Monolitik	Modüler, provider tabanlı
LINQ desteği	Var	Var (daha optimize)
Migration	EF Migrations	EF Core Migrations (Add-Migration, Update-Database)
Geliştirme	Aktif değil	Aktif

2.3 Bölüm Özeti

- HastaneMVC projesi EF Core kullanır çünkü .NET 8 üzerine kuruludur.
- Migration komutları EF Core'da Package Manager Console üzerinden çalıştırılır.

3. Üç Katmanlı Mimari

3.1 Genel Bakış

Üç katmanlı mimari, uygulamayı üç ayrı sorumluluk alanına bölen tasarım yaklaşımıdır:

1. **Sunum Katmanı (Presentation Layer):** Kullanıcı arayüzü. ASP.NET Core MVC'de Controllers + Views.
2. **İş Mantığı Katmanı (Business Logic Layer):** İş kuralları, hesaplamalar, doğrulamalar. HastaneMVC'de `Services` klasörü.
3. **Veri Erişim Katmanı (Data Access Layer):** Veritabanı işlemleri. HastaneMVC'de `Data/HastaneContext.cs` ve EF Core.

Bu ayırım sayesinde her katman bağımsız olarak değiştirilebilir; veritabanı sağlayıcısı değiştirilse Controller kodu değişmez.

3.2 HastaneMVC'de Karşılığı

```
Sunum:      Controllers/, Views/
İş Mantığı: Services/
Veri Erişim: Data/HastaneContext.cs, Migrations/
```

3.3 N-Tier Architecture

N-katmanlı mimari, üç katmanlı mimarinin genelleştirilmiş halidir. Service katmanı altında Repository, üzerinde API katmanı eklenebilir. HastaneMVC'de Service Layer örüntüsü uygulanmıştır (`BaseService` , `RandevuService` , `DoktorService`).

4. LINQ

4.1 Genel Bakış

LINQ (Language Integrated Query), C# içinde SQL benzeri sorgu yazmayı sağlayan dil özelliğidir. EF Core ile birlikte kullanıldığında LINQ ifadeleri SQL sorgularına çevrilir.

4.2 Sözdizimi

```
// Method syntax (proje boyunca kullanılan)
var sonuc = koleksiyon.Where(...).OrderBy(...).Select(...).ToList();
```

4.3 SQL Karşılıkları

LINQ	SQL Karşılığı
<code>Where(x => x.Id == 5)</code>	<code>WHERE Id = 5</code>
<code>OrderBy(x => x.AdSoyad)</code>	<code>ORDER BY AdSoyad</code>
<code>OrderByDescending(...)</code>	<code>ORDER BY ... DESC</code>
<code>Select(x => new { ... })</code>	<code>SELECT ...</code>
<code>FirstOrDefault()</code>	<code>SELECT TOP 1</code>
<code>Count()</code>	<code>COUNT(*)</code>
<code>Any(x => x.IsDeleted == false)</code>	<code>EXISTS (...)</code>
<code>Include(x => x.Doktor)</code>	<code>JOIN</code>
<code>ThenInclude(...)</code>	<code>İç içe JOIN</code>

4.4 Örnekler

Örnek 4.4.1 — Where + Select ile DTO'ya dönüştürme

```
// HastaneMVC/Controllers/BransController.cs
public IActionResult Index()
{
    var bransListesiDTO = _context.Branslar
        .Where(i => i.IsDeleted == false)
        .Select(b => new BransModelDTO
        {
            Id = b.Id,
            BransAdi = b.BransAdi
        })
        .ToList();

    return View(bransListesiDTO);
}
```

Çıktı: (yok — yan etki: aktif branş kayıtları DTO listesi olarak view'e gönderilir.)

Üretilen SQL (yaklaşık):

```
SELECT [b].[Id], [b].[BransAdi]
FROM [Branslar] AS [b]
WHERE [b].[IsDeleted] = CAST(0 AS bit)
```

Örnek 4.4.2 — Where + Include + ThenInclude ile JOIN

```
// HastaneMVC/Services/RandevuService.cs
var randevular = _context.Randevular
    .Include(r => r.Doktor)
    .ThenInclude(d => d.Brans)
    .Include(r => r.Hasta)
    .Where(r => r.IsDeleted == false)
    .ToList();
```

Çıktı: (yok — yan etki: Randevu + Doktor + Brans + Hasta tabloları JOIN'lenerek tek sorguda yüklenir.)

Örnek 4.4.3 — Any ile varlık kontrolü

```
// HastaneMVC/Controllers/HastaController.cs (parça)
var mevcutHasta = _context.Hastalar
    .Any(h => h.TcNo == yeniHasta.TcNo && h.IsDeleted == false);

if (mevcutHasta)
{
    ModelState.AddModelError("TcNo", "Bu TC No zaten kayıtlı!");
    return View(yeniHasta);
}
```

Çıktı: (yok — yan etki: aynı TC No'ya sahip kayıt varsa ModelState'e hata eklenir ve form geri döndürülür.)

4.5 API Referansı

Metot	Açıklama	Dönüş
<code>Where(predicate)</code>	Filtreler	<code>IEnumerable<T></code>
<code>Select(selector)</code>	Projeksiyon	<code>IEnumerable<TResult></code>
<code>OrderBy(keySelector)</code>	Artan sıralama	<code>IOrderedEnumerable<T></code>
<code>FirstOrDefault(predicate?)</code>	İlk öge veya <code>null</code>	<code>T</code>
<code>Find(key)</code>	Primary key ile bulur	<code>T</code>
<code>Any(predicate)</code>	Eşleşen var mı?	<code>bool</code>
<code>Count(predicate?)</code>	Eşleşen sayısı	<code>int</code>
<code>Include(navigation)</code>	İlişkili veriyi yükler (eager loading)	<code>IIncludableQueryable<T,P></code>
<code>ToList()</code>	Sorguyu çalıştırıp listeye çevirir	<code>List<T></code>

4.6 Yaygın Hatalar

4.6.1 — `null.ToUpper()` çağırmak

Hatalı kod:

```
// HastaneMVC/Services/DoktorService.cs (yanlış versiyon)
if (ara.ToUpper() != null)
{
    // ...
}
```

Hata mesajı:

```
System.NullReferenceException: Object reference not set to an instance of an object.
```

Sebebi: `ara` parametresi `null` ise `null.ToUpper()` çağrısı yapılamaz; metot çağrısı önce, null kontrolü sonra yapıldığı için exception fırlatılır.

Düzeltilmiş kod:

```
// HastaneMVC/Services/DoktorService.cs
if (ara != null)
{
    doktorlistesi = _context.Doktorlar
        .Where(d => d.Brans.BransAdi == ara.ToUpper() && d.IsDeleted == false)
        .Select(d => new DoktorModel { /* ... */ })
        .OrderBy(i => i.AdSoyad)
        .ToList();
}
else
{
    doktorlistesi = _context.Doktorlar
        .Where(d => d.IsDeleted == false)
        .Select(d => new DoktorModel { /* ... */ })
        .ToList();
}
```

4.6.2 — `Where(...).GroupBy(...)` sonucunu DTO listesi olarak view'e göndermek

Sebebi: View `IEnumerable<BransModelDTO>` beklerken `IEnumerable<IGrouping<...>>` gönderilirse tip uyumsuzluğu nedeniyle runtime exception oluşur.

Düzeltilmiş yaklaşım: `GroupBy` sonucunu `Select` ile DTO'ya çevirip `ToList()` ile materyalize et.

4.7 Mini Sözlük

- **Eager loading:** İlişkili verinin `Include` ile birlikte aynı sorguda yüklenmesi.
- **Lazy loading:** İlişkili verinin ilk erişildiği anda ayrı sorguyla yüklenmesi.
- **Projeksiyon (projection):** `Select` ile bir tipten başka bir tipe dönüştürme.
- **Predicate:** Boolean dönen lambda ifadesi (`x => x.Id > 5`).
- **Materyalizasyon:** Sorgunun `ToList` , `ToArray` , `FirstOrDefault` gibi metotlarla gerçekten çalıştırılıp veri çekmesi.

5. DTO ve ViewModel

5.1 Genel Bakış

DTO (Data Transfer Object): Katmanlar arasında veri taşımak için kullanılan, davranış içermeyen sade nesne. HastaneMVC'de DTO'lar Controller ↔ View arasında ve Service ↔ Controller arasında veri taşır. Entity'nin kendisi view'e gönderilmez çünkü:

- Entity'de gereksiz alanlar bulunabilir.
- Entity'de navigation property'ler vardır; form'dan gelmedikleri için validation hatasına yol açarlar.
- Güvenlik: hassas alanlar (`IsDeleted` , `DeleteTime`) view'e taşınmamalı.

ViewModel (VM): Bir view'in birden fazla farklı veri kaynağına ihtiyaç duyduğu durumda kullanılan birleşik model. Örneğin Randevu ekleme formu hem yeni randevu bilgisini, hem doktor listesini, hem hasta listesini, hem branş listesini ister; bu dört bilgi **RandevuEkleVM** içinde tek nesnede toplanır.

5.2 Fark Tablosu

Konu	DTO	ViewModel
Amaç	Veri taşıma	View'in birleşik modeli
İçerik	Tek bir entity'nin alanları	Birden çok DTO + ek alanlar
Konum	DTOs/ klasörü	ViewModels/ klasörü
Kullanım	Listeleme, basit form	Karmaşık form, dropdown'lu sayfa

5.3 Örnekler

Örnek 5.3.1 — Basit DTO

```
// HastaneMVC/DTOs/BransModelDTO.cs
using System.ComponentModel.DataAnnotations;

namespace HastaneMVC.DTOs
{
    public class BransModelDTO
    {
        public int Id { get; set; }

        [Required(ErrorMessage = "Zorunlu alan")]
        [StringLength(150)]
        public string BransAdi { get; set; }
    }
}
```

Örnek 5.3.2 — ViewModel

```
// HastaneMVC/ViewModels/RandevuEkleVM.cs
using HastaneMVC.DTOs;
using Microsoft.AspNetCore.Mvc.ModelBinding.Validation;

namespace HastaneMVC.ViewModels
{
    public class RandevuEkleVM
    {
        public RandevuModelDTO YeniRandevu { get; set; }

        [ValidateNever]
        public IEnumerable<HastaModelDTO> HastaListesi { get; set; }

        [ValidateNever]
        public IEnumerable<DoktorModelDTO> DoktorListesi { get; set; }

        [ValidateNever]
        public IEnumerable<BransModelDTO> BransListesi { get; set; }
    }
}
```

Çıktı: (yok — yan etki: **RandevuEkleVM** view'e gönderildiğinde dropdown'lar dolu gelir; **[ValidateNever]** sayesinde liste alanları ModelState kontrolüne dahil edilmez.)

5.4 Mini Sözlük

- **DTO**: Davranışsız, sadece veri içeren transfer nesnesi.
- **ViewModel**: View'e özel hazırlanmış, birden çok kaynağı birleştiren model.
- **[ValidateNever]** : Bir property'nin ModelState doğrulamasından muaf tutulmasını sağlayan attribute (**Microsoft.AspNetCore.Mvc.ModelBinding.Validation** namespace'i).

5.5 Bölüm Özeti

- View'e Entity yerine DTO veya ViewModel gönderilir.
- Birden çok kaynak gerekiyorsa ViewModel kullanılır.
- ViewModel'in liste alanlarına **[ValidateNever]** eklenir, aksi halde ModelState **false** döner.

6. Tag Helpers

6.1 Genel Bakış

Tag Helper, Razor view'lerde HTML elementlerine **asp-** öneki ile öznitelik eklemeyi sağlayan ASP.NET Core özelliğidir. Sunucu tarafında bu öznitelikler işlenip standart HTML'e dönüştürülür.

6.2 Sık Kullanılan Tag Helper'lar

Tag Helper	Görevi	Örnek
<code>asp-controller</code>	Hedef controller	<code>asp-controller="Brans"</code>
<code>asp-action</code>	Hedef action metot	<code>asp-action="Ekle"</code>
<code>asp-route-id</code>	Route parametresi	<code>asp-route-id="@Brans.Id"</code>
<code>asp-for</code>	Model property'ye bağlama	<code>asp-for="BransAdi"</code>
<code>asp-validation-for</code>	Validation hatasını gösterme	<code>asp-validation-for="BransAdi"</code>
<code>asp-items</code>	Select için liste	<code>asp-items="@ (new SelectList(Model.BransListesi, "Id", "BransAdi"))"</code>

6.3 Örnekler

Örnek 6.3.1 — Form ve input bağlama

```
<!-- HastaneMVC/Views/Brans/Ekle.cshtml (parça) -->
<form asp-action="Ekle" method="post">
  <div class="mb-4">
    <label asp-for="BransAdi" class="form-label fw-bold">Brans Adı</label>
    <input asp-for="BransAdi" class="form-control form-control-lg"
      placeholder="Örn: Kardiyoloji, Göz Hastalıkları..." />
    <span asp-validation-for="BransAdi" class="text-danger"></span>
  </div>
  <button type="submit" class="btn btn-success">Bransı Kaydet</button>
</form>
```

Çıktı (üretilen HTML, yaklaşık):

```
<form action="/Brans/Ekle" method="post">
  <div class="mb-4">
    <label for="BransAdi" class="form-label fw-bold">Brans Adı</label>
    <input type="text" id="BransAdi" name="BransAdi"
      class="form-control form-control-lg"
      placeholder="Örn: Kardiyoloji, Göz Hastalıkları..."
      data-val="true"
      data-val-required="Zorunlu alan" />
    <span class="text-danger field-validation-valid"
      data-valmsg-for="BransAdi" data-valmsg-replace="true"></span>
  </div>
  <button type="submit" class="btn btn-success">Bransı Kaydet</button>
</form>
```

Örnek 6.3.2 — Dropdown listesi

```
<!-- HastaneMVC/Views/Randevu/Ekle.cshtml (parça) -->
<select asp-for="YeniRandevu.HastaId"
        asp-items="@((new SelectList(Model.HastaListesi, "Id", "AdSoyad")))"
        class="form-select">
    <option value="">-- Lütfen Hasta Seçiniz --</option>
</select>
```

Çıktı: (yok — yan etki: `HastaListesi` koleksiyonundaki her öge için `<option value="Id">AdSoyad</option>` üretilir.)

6.4 Bölüm Özeti

- Tag Helper'lar HTML üretimini server tarafında düzenler ve model bağlamayı otomatikleştirir.
- `asp-validation-for` yalnızca **mesajı gösterir**; validation kuralı modelde `[Required]` gibi attribute'lerle tanımlanır.

7. AJAX ve jQuery

7.1 Genel Bakış

AJAX (Asynchronous JavaScript and XML), bir web sayfasının tamamı yenilenmeden sunucudan veri çekip sayfanın bir bölümünü güncellemesini sağlar. HastaneMVC'de randevu sayfalarında branş seçildiğinde, sadece o branşa bağlı doktor listesi AJAX ile çekilir; sayfanın geri kalanı yenilenmez.

7.2 Sözdizimi

```
$.ajax({
    url: '/Controller/Action',
    type: 'GET',
    data: { parametre: deger },
    success: basariliCagrildigindaCalisacakFonksiyon,
    error: hataOldugundaCalisacakFonksiyon
});
```

7.3 Örnekler

Örnek 7.3.1 — Branş seçilince doktor listesini getirme (Ekle sayfası)

```

<!-- HastaneMVC/Views/Randevu/Ekle.cshtml (script bölümü) -->
@section Scripts {
    @{
        await Html.RenderPartialAsync("_ValidationScriptsPartial");
    }

    <script>
        $(document).ready(function () {
            $('#bransSelect').change(BransDegisti);
        });

        function BransDegisti() {
            var secilenBransId = $('#bransSelect').val();
            if (secilenBransId != "") {
                AjaxIleDoktorlariGetir(secilenBransId);
            } else {
                DoktorKutusunuKilitle();
            }
        }

        function AjaxIleDoktorlariGetir(bransId) {
            $.ajax({
                url: '/Randevu/DoktorlariGetir',
                type: 'GET',
                data: { bransId: bransId },
                success: DoktorKutusunuDoldur,
                error: SunucuHatasi
            });
        }

        function DoktorKutusunuDoldur(gelenVeri) {
            var doktorSelect = $('#doktorSelect');
            doktorSelect.empty();
            doktorSelect.append('<option value="">-- Lütfen Doktor Seçiniz --</option>');
            for (var i = 0; i < gelenVeri.length; i++) {
                var doktor = gelenVeri[i];
                doktorSelect.append('<option value="' + doktor.id + '">' + doktor.adSoyad + '</option>');
            }
        }

        function DoktorKutusunuKilitle() {
            var doktorSelect = $('#doktorSelect');
            doktorSelect.empty();
            doktorSelect.append('<option value="">-- Lütfen Önce Branş Seçiniz --</option>');
        }

        function SunucuHatasi() {
            alert("Sunucu hatası oluştu!");
        }
    </script>
}

```

Sunucu tarafı:

```
// HastaneMVC/Controllers/RandevuController.cs
[HttpGet]
public JsonResult DoktorlariGetir(int bransId)
{
    var doktorlar = _context.Doktorlar
        .Where(d => d.BransId == bransId && d.IsDeleted == false)
        .Select(d => new
        {
            id = d.Id,
            adSoyad = d.AdSoyad
        })
        .ToList();

    return Json(doktorlar);
}
```

Çıktı (tarayıcı → sunucu isteği):

```
GET /Randevu/DoktorlariGetir?bransId=11
```

Çıktı (sunucu yanıtı):

```
[
  { "id": 3, "adSoyad": "Dr. Enes" },
  { "id": 7, "adSoyad": "Selim" }
]
```

Örnek 7.3.2 — Güncelleme sayfasında mevcut branşı seçili gösterme

Güncelleme sayfası açıldığında randevunun mevcut branşı seçili gelmelidir. Bunun için bir gizli input ile branş ID'si taşınır, sayfa yüklendiğinde JavaScript bu ID'yi okuyup dropdown'a set eder ve AJAX ile o branşın doktor listesini çeker.

```
<!-- HastaneMVC/Views/Randevu/Guncelle.cshtml (parça) -->
<input type="hidden" id="gizliBransId" value="@Model.YeniRandevu.BransId" />

<select id="bransSelect"
    asp-items="@((new SelectList(Model.BransListesi, "Id", "BransAdi")))">
    <option value="">-- Branş Seçiniz --</option>
</select>

<select asp-for="YeniRandevu.DoktorId" id="doktorSelect"
    asp-items="@((new SelectList(Model.DoktorListesi, "Id", "AdSoyad")))">
    <option value="">-- Doktor Seçiniz --</option>
</select>
```

```

<!-- HastaneMVC/Views/Randevu/Guncelle.cshtml (script bölümü) -->
@section Scripts {
    @{
        await Html.RenderPartialAsync("_ValidationScriptsPartial");
    }

    <script>
        $(document).ready(function () {
            $('#bransSelect').change(BransDegisti);
            var mevcutBransId = $('#gizliBransId').val();
            $('#bransSelect').val(mevcutBransId);
            AjaxIleDoktorlariGetir(mevcutBransId);
        });

        function BransDegisti() {
            var secilenBransId = $('#bransSelect').val();
            if (secilenBransId != "") {
                AjaxIleDoktorlariGetir(secilenBransId);
            } else {
                DoktorKutusunuKilitle();
            }
        }

        function AjaxIleDoktorlariGetir(bransId) {
            $.ajax({
                url: '/Randevu/DoktorlariGetir',
                type: 'GET',
                data: { bransId: bransId },
                success: DoktorKutusunuDoldur,
                error: SunucuHatasi
            });
        }

        function DoktorKutusunuDoldur(gelenVeri) {
            var doktorSelect = $('#doktorSelect');
            doktorSelect.empty();
            doktorSelect.append('<option value="">-- Lütfen Doktor Seçiniz --</option>');
            for (var i = 0; i < gelenVeri.length; i++) {
                var doktor = gelenVeri[i];
                doktorSelect.append('<option value="' + doktor.id + '">' + doktor.adSoyad + '</option>');
            }
        }

        function SunucuHatasi() {
            alert("Sunucu hatası oluştu!");
        }
    </script>
}

```

Çıktı: (yok — yan etki: Güncelle sayfası açıldığında mevcut branş otomatik seçilir, o branşa ait doktorlar dropdown'a yüklenir.)

7.4 Yaygın Hatalar

7.4.1 — `<script>` bloğunun `@section Scripts` dışında yazılması

Hatalı kod:

```
<!-- yanlış: section dışında script -->
@section Scripts {
    @{
        await Html.RenderPartialAsync("_ValidationScriptsPartial");
    }
}

<script>
    $(document).ready(function () { /* ... */ });
</script>
```

Hata mesajı (tarayıcı konsolu):

```
Uncaught SyntaxError: Unexpected end of input
Uncaught ReferenceError: AjaxIleDoktorlariGetir is not defined
```

Sebebi: `@section Scripts` bloğu Layout tarafından render edilen yere yazılır. Section dışındaki `<script>` body'de farklı bir konumda render edilir ve Layout'taki section sayıcı kapanış parantezini eksik bulur; bu durum syntax hatasına yol açar.

Düzeltilmiş kod: `<script>` bloğu `@section Scripts { ... }` içine alınmalı.

7.4.2 — Yardımcı fonksiyonun başka fonksiyon içinde tanımlanması

Hatalı kod:

```
function DoktorKutusunuDoldur(gelenVeri) {
    // ...
    function SunucuHatasi() {
        alert("Sunucu hatası");
    }
}
// AJAX error: SunucuHatasi → bu noktada SunucuHatasi tanımsız
```

Hata mesajı:

```
Uncaught ReferenceError: SunucuHatasi is not defined
```

Sebebi: JavaScript'te iç fonksiyonlar yalnızca dış fonksiyonun kapsamı içinde görünür. AJAX'ın `error` callback'i global scope'tan çağrıldığı için iç fonksiyonu göremez.

Düzeltilmiş kod: `SunucuHatasi` fonksiyonu `DoktorKutusunuDoldur` 'un dışına alınmalı.

7.5 Mini Sözlük

- **AJAX:** Sayfayı yenilemeden sunucudan veri çekme tekniği.

- **JSON:** JavaScript Object Notation, AJAX yanıtlarının taşındığı metin formatı.
- **JsonResult** : ASP.NET Core'da AJAX yanıtı olarak JSON döndürmeye yarayan action result tipi.
- **success callback:** AJAX isteği başarılı olduğunda çağrılan fonksiyon.
- **\$(document).ready()** : DOM tamamen yüklendikten sonra çalışacak kodu sarmalayan jQuery fonksiyonu.

8. Model Validation

8.1 Genel Bakış

Model Validation, formdan gelen verinin iş kurallarına uygun olup olmadığını sunucu ve istemci tarafında kontrol etmeye yarar. ASP.NET Core MVC'de bu iş Data Annotations attribute'leri ile yapılır. Validation iki katmanda çalışır:

1. **İstemci tarafı (client-side):** jQuery validate ile anlık doğrulama; kullanıcıdan kaynaklı yanlış girişler form gönderilmeden gösterilir.
2. **Sunucu tarafı (server-side):** `ModelState.IsValid` ile kontrol; istemci kontrolü atlanırsa bile sunucu kabul etmez.

İkisi birlikte kullanılır: istemci kontrolü kullanıcı deneyimi için, sunucu kontrolü güvenlik için zorunludur.

8.2 Sözdizimi

```
[Required(ErrorMessage = "Zorunlu alan")]
[StringLength(150, ErrorMessage = "En fazla 150 karakter olabilir")]
public string BransAdi { get; set; }
```

8.3 Sık Kullanılan Attribute'ler

Attribute	Görev
<code>[Required(ErrorMessage="...")]</code>	Boş olamaz
<code>[StringLength(150)]</code>	Maksimum karakter
<code>[StringLength(11, MinimumLength=11)]</code>	Sabit uzunluk
<code>[Range(1, 100)]</code>	Sayısal aralık
<code>[EmailAddress]</code>	E-posta formatı
<code>[ValidateNever]</code>	Bu alanı doğrulama

8.4 Örnekler

Örnek 8.4.1 — DTO'ya validation ekleme

```
// HastaneMVC/DTOs/HastaModelDTO.cs
using System.ComponentModel.DataAnnotations;

namespace HastaneMVC.DTOs
{
    public class HastaModelDTO
    {
        public int Id { get; set; }

        [Required(ErrorMessage = "Ad Soyad zorunludur")]
        [StringLength(150)]
        public string AdSoyad { get; set; }

        [Required(ErrorMessage = "TC No zorunludur")]
        [StringLength(11, MinimumLength = 11, ErrorMessage = "TC No 11 karakter olmalıdır")]
        public string TcNo { get; set; }
    }
}
```

Örnek 8.4.2 — Controller'da ModelState kontrolü

```
// HastaneMVC/Controllers/BransController.cs
[HttpPost]
public IActionResult Ekle(BransModelDTO modelDTO)
{
    if (ModelState.IsValid)
    {
        var entity = new BransModel { BransAdi = modelDTO.BransAdi };
        _context.Branslar.Add(entity);
        _context.SaveChanges();
        TempData["success"] = "Branş başarıyla eklendi!";
        return RedirectToAction("Index");
    }
    return View(modelDTO);
}
```

Çıktı: (yok — yan etki: ModelState geçerliyse kayıt eklenir ve Toastr'a "success" mesajı bırakılır; geçersizse form aynı verilerle geri döner ve `asp-validation-for` mesajları görüntülenir.)

Örnek 8.4.3 — View'de validation script'lerini etkinleştirme

```
<!-- HastaneMVC/Views/Brans/Ekle.cshtml (parça) -->
<input asp-for="BransAdi" class="form-control" />
<span asp-validation-for="BransAdi" class="text-danger"></span>

@section Scripts {
    @{
        await Html.RenderPartialAsync("_ValidationScriptsPartial");
    }
}
```

Çıktı (kullanıcı boş bırakıp Kaydet'e bastığında):

Zorunlu alan

8.5 Yaygın Hatalar

8.5.1 — HTML `required` attribute'unun İngilizce mesaj göstermesi

Hatalı kod:

```
<input asp-for="BransAdi" required />
```

Çıktı (tarayıcı tarafından):

The BransAdi field is required.

Sebebi: HTML'in yerleşik `required` attribute'u tarayıcının kendi validasyonunu tetikler ve `ErrorMessage` değerini okumaz. Tarayıcı mesajları İngilizce gelir.

Düzeltilmiş kod: HTML'deki `required` 'ı kaldır, DTO'da `[Required(ErrorMessage="Zorunlu alan")]` kullan ve view'e `_ValidationScriptsPartial` 'i ekle.

8.5.2 — Navigation property'ye `[Required]` koymak

Hatalı kod:

```
public class RandevuModel : BaseEntity
{
    public int DoktorId { get; set; }

    [Required] // YANLIŞ
    public DoktorModel Doktor { get; set; }
}
```

Sebebi: Navigation property formdan gelmez; ModelState her zaman `false` döner ve form kaydedilemez.

Düzeltilmiş kod: Navigation property'lere `[Required]` yazılmaz; sadece foreign key (`DoktorId`) zorunlu yapılır veya ViewModel içinde `[ValidateNever]` kullanılır.

8.5.3 — DTO'da listeleme alanına `[Required]` koymak

Hatalı kod:

```
public class RandevuModelDTO
{
    [Required] // YANLIŞ – bu alan sadece listelemede dolar
    public string HastaAdSoyad { get; set; }
}
```

Sebebi: `HastaAdSoyad` formdan gelmez (form sadece `HastaId` taşır); ModelState bu alanı zorunlu görünce hata fırlatır.

Düzeltilmiş kod: Sadece formdan gelmesi gereken alanlara `[Required]` koyulur.

8.6 Bölüm Özeti

- Validation modelde tanımlanır, controller'da `ModelState.IsValid` ile kontrol edilir, view'de `asp-validation-for` ile gösterilir.
- HTML `required` Türkçe mesaj göstermez; `[Required(ErrorMessage="...")]` + `_ValidationScriptsPartial` gerekir.
- Navigation property ve listeleme-amaçlı alanlara `[Required]` yazılmaz.

9. Toastr Bildirimleri

9.1 Genel Bakış

Toastr, ekranın köşesinde belirip kısa süre sonra kaybolan bildirim kutucukları sağlayan jQuery eklentisidir. HastaneMVC'de Toastr, CRUD işlemleri sonrası kullanıcıya başarı/hata bildirmek için kullanılır. Mesajlar `TempData` üzerinden controller'dan view'e taşınır.

9.2 Kurulum (CDN)

```
<!-- HastaneMVC/Views/Shared/_Layout.cshtml (head) -->
<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/toastr.js/latest/toastr.min.css" />
```

9.3 Partial View

```
<!-- HastaneMVC/Views/Shared/Toast.cshtml -->
@if (TempData["success"] != null)
{
    <script src="~/lib/jquery/dist/jquery.min.js"></script>
    <script src="https://cdnjs.cloudflare.com/ajax/libs/toastr.js/latest/toastr.min.js"></script>
    <script type="text/javascript">
        toastr.success('@TempData["success"]');
    </script>
}

@if (TempData["error"] != null)
{
    <script src="~/lib/jquery/dist/jquery.min.js"></script>
    <script src="https://cdnjs.cloudflare.com/ajax/libs/toastr.js/latest/toastr.min.js"></script>
    <script type="text/javascript">
        toastr.error('@TempData["error"]');
    </script>
}
```

9.4 Layout'ta Çağırma

```
<!-- HastaneMVC/Views/Shared/_Layout.cshtml (body içinde) -->
<div class="container container-main">
    <main role="main" class="pb-3">
        <partial name="Toast" />
        @RenderBody()
    </main>
</div>
```

9.5 Controller'da TempData Yazma

```
// HastaneMVC/Controllers/BransController.cs
[HttpPost]
public IActionResult Ekle(BransModelDTO modelDTO)
{
    if (ModelState.IsValid)
    {
        try
        {
            var entity = new BransModel { BransAdi = modelDTO.BransAdi };
            _context.Branslar.Add(entity);
            _context.SaveChanges();
            TempData["success"] = "Branş başarıyla eklendi!";
        }
        catch (Exception)
        {
            TempData["error"] = "Branş eklenirken bir hata oluştu!";
        }
        return RedirectToAction("Index");
    }
    return View(modelDTO);
}
```

Çıktı: (yok — yan etki: kayıt başarılıysa Index sayfası açıldığında sağ üst köşede yeşil "Branş başarıyla eklendi!" bildirimi belirir; hata varsa kırmızı bildirim çıkar.)

9.6 Konum Seçenekleri

Toastr default olarak `toast-top-right` konumunda görünür. Değiştirmek için:

```
toastr.options = {
    positionClass: "toast-top-right",
    timeout: 3000,
    closeButton: true,
    progressBar: true
};
```

Diğer konum sınıfları: `toast-top-left`, `toast-bottom-right`, `toast-bottom-left`, `toast-top-center`, `toast-bottom-center`.

9.7 Yaygın Hatalar

9.7.1 — TempData anahtarının büyük/küçük harf uyumsuzluğu

Hatalı kod:

```
TempData["success"] = "Eklendi"; // küçük harf
```

```
@if (TempData["Success"] != null) { /* büyük harf – eşleşmez */ }
```

SebeP: `TempData` anahtar adında büyük/küçük harfe duyarlıdır. İki yerde tutarsız yazım yanıtın hiç gösterilmemesine yol açar.

Düzeltilmiş kod: Tüm controller'larda ve `Toast.cshtml` 'de aynı yazım kullanılmalı (proje boyunca tutarlı olarak küçük harf seçilmiştir: `"success"` , `"error"`).

9.7.2 — Partial view dosyasının `Views/Shared/` altında olmaması

Hata mesajı:

```
InvalidOperationException: The partial view 'Toast' was not found. The following locations were searched:
/Views/Home/Toast.cshtml
/Views/Shared/Toast.cshtml
```

SebeP: ASP.NET Core önce mevcut controller'ın view klasörünü, sonra `Shared` 'i tarar. Dosya başka konumdaysa bulunamaz.

Düzeltilmiş kod: `Toast.cshtml` dosyası `Views/Shared/` klasörüne yerleştirilmeli.

9.8 Bölüm Özeti

- Toastr CDN üzerinden Layout'a eklenir.
- `TempData` controller'dan view'e tek seferlik mesaj taşır.
- `Toast.cshtml` partial view'ı Layout'tan çağrılır; her sayfada otomatik çalışır.

10. Soft Delete

10.1 Genel Bakış

Soft Delete, bir kaydın veritabanından fiziksel olarak silinmek yerine "silindi" olarak işaretlenmesidir. HastaneMVC'de tüm entity'ler `BaseEntity` aracılığıyla `IsDeleted` (bool) ve `DeleteTime` (DateTime?) alanlarına sahiptir. Silme işlemi bu alanları günceller; listeleme sorgularına `Where(x => x.IsDeleted == false)` eklenir.

Bu yaklaşımın avantajı: yanlışlıkla silinen kayıtlar geri getirilebilir, denetim izleri korunur, ilişkili veriler (foreign key'ler) korunur.

10.2 BaseEntity Tanımı

```
// HastaneMVC/Models/BaseEntity.cs
using System.ComponentModel.DataAnnotations;

namespace HastaneMVC.Models
{
    public abstract class BaseEntity
    {
        [Key]
        public int Id { get; set; }
        public bool IsDeleted { get; set; } = false;
        public DateTime CreateTime { get; private set; } = DateTime.Now;
        public DateTime? DeleteTime { get; set; }
    }
}
```

10.3 Soft Delete Action

```
// HastaneMVC/Controllers/BransController.cs
[HttpPost, ActionName("Sil")]
public IActionResult SilOnaylandi(int id)
{
    var brans = _context.Branslar.Find(id);
    if (brans != null)
    {
        brans.IsDeleted = true;
        brans.DeleteTime = DateTime.Now;
        _context.SaveChanges();
        TempData["success"] = "Branş silindi!";
    }
    return RedirectToAction("Index");
}
```

Çıktı: (yok — yan etki: kayıt veritabanında kalır ama `IsDeleted=true` olur ve `DeleteTime` o anın zaman damgasıyla doldurulur.)

10.4 Listeleme Sorgusunda Filtreleme

```
// HastaneMVC/Controllers/BransController.cs
public IActionResult Index()
{
    var bransListesi = _context.Branslar
        .Where(i => i.IsDeleted == false)
        .Select(b => new BransModelDTO
        {
            Id = b.Id,
            BransAdi = b.BransAdi
        })
        .ToList();

    return View(bransListesi);
}
```

Çıktı: (yok — yan etki: yalnızca silinmemiş branşlar listelenir.)

10.5 Mini Sözlük

- **Soft Delete:** Veriyi silinmiş olarak işaretleme; fiziksel silme yapılmaz.
- **Hard Delete:** Kaydı veritabanından tamamen kaldırma (`_context.Remove(entity)`).
- **Tombstone:** Silindi olarak işaretlenmiş ama hâlâ tabloda duran kayıt.

10.6 Bölüm Özeti

- `BaseEntity` üzerinden `IsDeleted` ve `DeleteTime` her entity'ye otomatik gelir.
- Silme actionlarında `Remove` yerine `IsDeleted = true` set edilir.
- Tüm listeleme LINQ sorgularına `Where(x => x.IsDeleted == false)` eklenir.

11. Middleware

11.1 Genel Bakış

Middleware, gelen HTTP isteği Controller'a ulaşmadan önce ve çıkan yanıt istemciye dönmeden önce çalışan ara katmanlardır. Her middleware bir sonrakini çağırmakla yükümlüdür; aksi halde pipeline kesilir. ASP.NET Core'da middleware'ler `Program.cs` içinde `app.Use...` sırasıyla tanımlanır ve sıraları önemlidir.

11.2 Pipeline Akışı

```
graph LR
    A[İstek] --> B[Middleware 1]
    B --> C[Middleware 2]
    C --> D[Controller]
    D --> C2[Middleware 2 dönüş]
    C2 --> B2[Middleware 1 dönüş]
    B2 --> E[Yanıt]
```

11.3 Yerleşik Middleware'ler

```
// HastaneMVC/Program.cs (kısmen)
var app = builder.Build();

app.UseHttpsRedirection(); // HTTP'yi HTTPS'e çevirir
app.UseStaticFiles();      // wwwroot'tan statik dosya sunar
app.UseRouting();          // URL'yi controller/action'a yönlendirir
app.UseAuthorization();    // yetki kontrolü
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
app.Run();
```

11.4 Inline Middleware

```
// HastaneMVC/Program.cs (örnek ekleme)
app.Use(async (context, next) =>
{
    Console.WriteLine($"İstek: {context.Request.Path}");
    await next();
    Console.WriteLine($"Yanıt: {context.Response.StatusCode}");
});
```

Çıktı (konsola):

```
İstek: /Brans/Index
Yanıt: 200
```

11.5 Sınıf Tabanlı Middleware

```
// HastaneMVC/Middleware/LoglamaMiddleware.cs
public class LoglamaMiddleware
{
    private readonly RequestDelegate _next;

    public LoglamaMiddleware(RequestDelegate next)
    {
        _next = next;
    }

    public async Task Invoke(HttpContext context)
    {
        var path = context.Request.Path;
        var method = context.Request.Method;
        Console.WriteLine($"[{DateTime.Now}] {method} {path}");

        await _next(context);
    }
}
```

```
// HastaneMVC/Program.cs
app.UseMiddleware<LoglamaMiddleware>();
```

Çıktı (konsola):

```
[28.04.2026 00:15:32] GET /Brans/Index
```

11.6 Bölüm Özeti

- Middleware'ler `Program.cs` 'te `app.Use...` ile sırayla tanımlanır.
- Her middleware `await _next(context)` ile bir sonrakini çağırır.
- Sınıf tabanlı middleware için `Invoke(HttpContext)` veya `InvokeAsync(HttpContext)` metodu zorunludur.

12. Base Sınıflar ve Kalıtım

12.1 Genel Bakış

HastaneMVC'de tekrar eden ortak davranışları azaltmak için `BaseEntity`, `BaseService`, `BaseController` gibi base sınıflar tanımlanmıştır. Türetilen sınıflar bu temel davranışları otomatik olarak alır.

12.2 BaseEntity

```
// HastaneMVC/Models/BaseEntity.cs
public abstract class BaseEntity
{
    [Key]
    public int Id { get; set; }
    public bool IsDeleted { get; set; } = false;
    public DateTime CreateTime { get; private set; } = DateTime.Now;
    public DateTime? DeleteTime { get; set; }
}
```

Her entity bundan türer:

```
// HastaneMVC/Models/BransModel.cs
public class BransModel : BaseEntity
{
    [Required(ErrorMessage = "Zorunlu alan")]
    [StringLength(150)]
    public string BransAdi { get; set; }

    public ICollection<DoktorModel> Doktorlar { get; set; }
}
```

12.3 BaseService

```
// HastaneMVC/Services/BaseService.cs
public abstract class BaseService
{
    protected readonly HastaneContext _context;

    public BaseService(HastaneContext context)
    {
        _context = context;
    }
}
```

```
// HastaneMVC/Services/DoktorService.cs
public class DoktorService : BaseService
{
    public DoktorService(HastaneContext context) : base(context)
    {
    }

    public List<DoktorModel> indexgetir(string ara)
    {
        List<DoktorModel> doktorlistesi;

        if (ara != null)
        {
            doktorlistesi = _context.Doktorlar
                .Where(d => d.Brans.BransAdi == ara.ToUpper() && d.IsDeleted == false)
                .Select(d => new DoktorModel
                {
                    Id = d.Id,
                    AdSoyad = d.AdSoyad,
                    BransId = d.BransId,
                    Brans = d.Brans
                })
                .OrderBy(i => i.AdSoyad)
                .ToList();
        }
        else
        {
            doktorlistesi = _context.Doktorlar
                .Where(d => d.IsDeleted == false)
                .Select(d => new DoktorModel
                {
                    Id = d.Id,
                    AdSoyad = d.AdSoyad,
                    BransId = d.BransId,
                    Brans = d.Brans
                })
                .ToList();
        }

        return doktorlistesi;
    }
}
```

12.4 BaseController

```
// HastaneMVC/Controllers/BaseController.cs
public class BaseController : Controller
{
    protected readonly HastaneContext _context;

    public BaseController(HastaneContext context)
    {
        _context = context;
    }

    protected IEnumerable<BransModelDTO> branslarigetir()
    {
        return _context.Branslar
            .Where(b => b.IsDeleted == false)
            .Select(c => new BransModelDTO
            {
                Id = c.Id,
                BransAdi = c.BransAdi
            })
            .ToList();
    }
}
```

Türetilen controller her ek servisini kendisi enjekte eder:

```
// HastaneMVC/Controllers/RandevuController.cs
public class RandevuController : BaseController
{
    private readonly RandevuService _randevuService;

    public RandevuController(HastaneContext context, RandevuService randevuService)
        : base(context)
    {
        _randevuService = randevuService;
    }
    // ...
}
```

12.5 Yaygın Hatalar

12.5.1 — Base sınıfta tüm türemiş sınıfların ihtiyacı olmayan servis istemek

Hatalı tasarım:

```
public BaseController(HastaneContext context, RandevuService randevuService, DoktorService doktorService)
{
    // ...
}
```

Sebebi: Tüm controller'lar `RandevuService` veya `DoktorService` kullanmak zorunda olmadığı için bu, gereksiz bağımlılık yaratır ve `BransController` gibi bunlara ihtiyacı olmayan controller'larda DI hatasına yol açabilir.

Düzeltilmiş yaklaşım: `BaseController` yalnızca tüm controller'ların ihtiyaç duyacağı `HastaneContext` 'i alır; her controller kendi servisini ek olarak alır ve `base(context)` ile temel sınıfı çağırır.

12.6 Bölüm Özeti

- `BaseEntity` her entity'ye `Id`, `IsDeleted`, `CreateTime`, `DeleteTime` ekler.
- `BaseService` ve `BaseController` ortak `_context` bağımlılığını merkezileştirir.
- Base sınıf yalnızca **gerçekten ortak olan** bağımlılıkları talep etmelidir.

13. Dependency Injection

13.1 Genel Bakış

Dependency Injection (Bağımlılık Enjeksiyonu), bir sınıfın ihtiyaç duyduğu bağımlılıkların dışarıdan (constructor üzerinden) verilmesi tasarım örüntüsüdür. ASP.NET Core, yerleşik bir DI container içerir; servisler `Program.cs` 'te kayıt edilir, controller constructor'larında otomatik olarak enjekte edilir.

13.2 Sözdizimi

```
// HastaneMVC/Program.cs
builder.Services.AddScoped<RandevuService>();
builder.Services.AddScoped<DoktorService>();

builder.Services.AddDbContext<HastaneContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("HastaneBaglantisi")));
```

13.3 Yaşam Süresi Seçenekleri

Metot	Yaşam Süresi
<code>AddScoped</code>	İstek başına bir kez oluşturulur
<code>AddTransient</code>	Her enjeksiyonda yeniden oluşturulur
<code>AddSingleton</code>	Uygulama ömrü boyunca tek instance

EF Core DbContext için `AddScoped` standarttır; `HastaneContext` ve servisler bu yaşam süresini kullanır.

13.4 Constructor Enjeksiyonu

```
// HastaneMVC/Controllers/BransController.cs
public class BransController : Controller
{
    private readonly HastaneContext _context;

    public BransController(HastaneContext context)
    {
        _context = context;
    }
    // ...
}
```

Çıktı: (yok — yan etki: ASP.NET Core, her istek için `HastaneContext` örneği üretilip constructor'a verir; controller manuel oluşturmaz.)

13.5 Bölüm Özeti

- Servisler `Program.cs` 'te kaydedilir; controller'lar constructor üzerinden alır.
- DbContext ve service'ler için `AddScoped` kullanılır.
- `private readonly` field ile saklanan bağımlılık, constructor dışında değiştirilemez.

14. .NET 8 Program.cs

14.1 Genel Bakış

.NET 6'dan itibaren `Startup.cs` kaldırılmıştır; tüm yapılandırma `Program.cs` içinde "minimal hosting" yaklaşımıyla yapılır. HastaneMVC bu yapıyı kullanır.

14.2 Tam Yapı

```
// HastaneMVC/Program.cs
using HastaneMVC.Data;
using HastaneMVC.Services;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

// Servis kayıtları (eskiden Startup.ConfigureServices)
builder.Services.AddControllersWithViews(options =>
{
    options.Filters.Add(new AutoValidateAntiforgeryTokenAttribute());
});

builder.Services.AddDbContext<HastaneContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("HastaneBaglantisi")));

builder.Services.AddScoped<RandevuService>();
builder.Services.AddScoped<DoktorService>();

var app = builder.Build();

// Middleware pipeline (eskiden Startup.Configure)
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

14.3 Eski vs Yeni

Eski (.NET 5 ve öncesi)	Yeni (.NET 6+)
Startup.cs + Program.cs	Tek Program.cs
ConfigureServices(IServiceCollection)	builder.Services.Add...
Configure(IApplicationBuilder, IWebHostEnvironment)	app.Use...

14.4 Bölüm Özeti

- .NET 8'de `Startup.cs` yoktur.
 - Tüm servis kayıtları `builder.Services` üzerinden yapılır.
 - Pipeline `var app = builder.Build();` çağrısından sonra `app.Use...` ile kurulur.
-

BÖLÜM II — HASTANEMVC PROJE ÖZELLİKLERİ

15. Veri Modeli ve Navigation Property

15.1 Entity İlişkileri

```
erDiagram
    BRANS ||--o{ DOKTOR : içerir
    DOKTOR ||--o{ RANDEVU : alır
    HASTA ||--o{ RANDEVU : alır
    RANDEVU ||--o{ RANDEVUTAHLIL : içerir
    TAHLILTUR ||--o{ RANDEVUTAHLIL : ait
```

15.2 Tam Model Tanımları

```
// HastaneMVC/Models/BransModel.cs
public class BransModel : BaseEntity
{
    [Required(ErrorMessage = "Zorunlu alan")]
    [StringLength(150)]
    public string BransAdi { get; set; }

    public ICollection<DoktorModel> Doktorlar { get; set; }
}
```

```
// HastaneMVC/Models/DoktorModel.cs
public class DoktorModel : BaseEntity
{
    [Required(ErrorMessage = "Zorunlu alan")]
    [StringLength(150)]
    public string AdSoyad { get; set; }

    public int BransId { get; set; }
    public BransModel Brans { get; set; }

    public ICollection<RandevuModel> Randevular { get; set; }
}
```

```
// HastaneMVC/Models/HastaModel.cs
public class HastaModel : BaseEntity
{
    [Required(ErrorMessage = "Ad Soyad zorunludur")]
    [StringLength(150)]
    public string AdSoyad { get; set; }

    [Required(ErrorMessage = "TC No zorunludur")]
    [StringLength(11, MinimumLength = 11)]
    public string TcNo { get; set; }

    public ICollection<RandevuModel> Randevular { get; set; }
}
```

```
// HastaneMVC/Models/RandevuModel.cs
public class RandevuModel : BaseEntity
{
    public int HastaId { get; set; }
    public HastaModel Hasta { get; set; }

    public int DoktorId { get; set; }
    public DoktorModel Doktor { get; set; }

    public DateTime TarihSaat { get; set; }
    public string DoktorNotu { get; set; }

    public ICollection<RandevuTahlilModel> RandevuTahlilleri { get; set; }
}
```

```
// HastaneMVC/Models/TahlilTurModel.cs
public class TahlilTurModel : BaseEntity
{
    [Required(ErrorMessage = "Zorunlu alan")]
    [StringLength(150)]
    public string Ad { get; set; }

    public ICollection<RandevuTahlilModel> RandevuTahlilleri { get; set; }
}
```

```
// HastaneMVC/Models/RandevuTahlilModel.cs
public class RandevuTahlilModel : BaseEntity
{
    public int RandevuId { get; set; }
    public RandevuModel Randevu { get; set; }

    public int TahlilTurId { get; set; }
    public TahlilTurModel TahlilTur { get; set; }

    public string Sonuc { get; set; }
}
```

15.3 DbContext

```
// HastaneMVC/Data/HastaneContext.cs
using HastaneMVC.Models;
using Microsoft.EntityFrameworkCore;

namespace HastaneMVC.Data
{
    public class HastaneContext : DbContext
    {
        public HastaneContext(DbContextOptions<HastaneContext> options) : base(options)
        {
        }

        public DbSet<BransModel> Branslar { get; set; }
        public DbSet<DoktorModel> Doktorlar { get; set; }
        public DbSet<HastaModel> Hastalar { get; set; }
        public DbSet<RandevuModel> Randevular { get; set; }
        public DbSet<TahlilTurModel> TahlilTurleri { get; set; }
        public DbSet<RandevuTahlilModel> RandevuTahlilleri { get; set; }

        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            modelBuilder.Entity<HastaModel>()
                .HasIndex(h => h.TcNo)
                .IsUnique();
        }
    }
}
```

15.4 appsettings.json

```
// HastaneMVC/appsettings.json
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "HastaneBaglantisi": "YOUR_CONNECTION_STRING"
  }
}
```

15.5 Bölüm Özeti

- Foreign key + navigation property birlikte tanımlanır (**BransId** + **Brans**).
- **ICollection<T>** ters yönlü ilişkiyi taşır.
- Unique constraint **OnModelCreating** içinde **HasIndex(...).IsUnique()** ile tanımlanır.

16. Branş Yönetimi (CRUD)

16.1 Amaç

Hastanenin branşlarını (Kardiyoloji, Göz Hastalıkları vb.) listeler, ekler, günceller ve soft delete ile siler. Branşlar doktorların gruplanmasında ve randevu sayfasındaki dropdown kaskadında kullanılır.

16.2 Controller — Tam Kod

```
// HastaneMVC/Controllers/BransController.cs
using Microsoft.AspNetCore.Mvc;
using HastaneMVC.Data;
using HastaneMVC.DTOS;
using HastaneMVC.Models;
using System.Linq;

namespace HastaneMVC.Controllers
{
    public class BransController : Controller
    {
        private readonly HastaneContext _context;

        public BransController(HastaneContext context)
        {
            _context = context;
        }

        public IActionResult Index()
        {
            var bransListesiDTO = _context.Branslar
                .Where(i => i.IsDeleted == false)
                .Select(b => new BransModelDTO
                {
                    Id = b.Id,
                    BransAdi = b.BransAdi
                })
                .ToList();

            return View(bransListesiDTO);
        }

        [HttpGet]
        public IActionResult Ekle()
        {
            return View();
        }

        [HttpPost]
        public IActionResult Ekle(BransModelDTO modelDTO)
        {
            if (ModelState.IsValid)
            {
                var entity = new BransModel { BransAdi = modelDTO.BransAdi.ToUpper() };
                _context.Branslar.Add(entity);
                _context.SaveChanges();
                TempData["success"] = "Brans başarıyla eklendi!";
                return RedirectToAction("Index");
            }
            return View(modelDTO);
        }

        [HttpGet]
        public IActionResult Guncelle(int id)
        {
            var brans = _context.Branslar.Find(id);

```

```

        var dto = new BransModelDTO
        {
            Id = brans.Id,
            BransAdi = brans.BransAdi
        };
        return View(dto);
    }

    [HttpPost]
    public IActionResult Guncelle(BransModelDTO guncelBrans)
    {
        if (ModelState.IsValid)
        {
            var eskiBrans = _context.Branslar.Find(guncelBrans.Id);
            eskiBrans.BransAdi = guncelBrans.BransAdi.ToUpper();

            _context.Branslar.Update(eskiBrans);
            _context.SaveChanges();
            TempData["success"] = "Brans güncellendi!";
            return RedirectToAction("Index");
        }
        return View(guncelBrans);
    }

    [HttpGet]
    public IActionResult Sil(int id)
    {
        var brans = _context.Branslar.Find(id);
        if (brans == null)
        {
            return NotFound();
        }

        var dto = new BransModelDTO
        {
            Id = brans.Id,
            BransAdi = brans.BransAdi
        };

        return View(dto);
    }

    [HttpPost, ActionName("Sil")]
    public IActionResult SilOnaylandi(int id)
    {
        var brans = _context.Branslar.Find(id);
        if (brans != null)
        {
            brans.IsDeleted = true;
            brans.DeleteTime = DateTime.Now;
            _context.SaveChanges();
            TempData["success"] = "Brans silindi!";
        }

        return RedirectToAction("Index");
    }
}

```

```

    }
}

```

16.3 Ekle View

```

<!-- HastaneMVC/Views/Brans/Ekle.cshtml -->
@using HastaneMVC.DT0s
@model BransModelDT0

@{
    ViewData["Title"] = "Yeni Branş Ekle";
}

<div class="container mt-5">
    <div class="row justify-content-center">
        <div class="col-md-6">
            <div class="card shadow border-0">
                <div class="card-header bg-success text-white py-3">
                    <h5 class="mb-0"><i class="bi bi-plus-circle-dotted"></i> Yeni Branş Kaydı</h5>
                </div>
                <div class="card-body p-4">
                    <form asp-action="Ekle" method="post">
                        <div class="mb-4">
                            <label asp-for="BransAdi" class="form-label fw-bold">Branş Adı</label>
                            <input asp-for="BransAdi" class="form-control form-control-lg"
                                placeholder="Örn: Kardiyoloji, Göz Hastalıkları..." />
                            <span asp-validation-for="BransAdi" class="text-danger"></span>
                        </div>

                        <hr />

                        <div class="d-grid gap-2 d-md-flex justify-content-md-end mt-4">
                            <a asp-action="Index" class="btn btn-outline-secondary px-4">
                                <i class="bi bi-arrow-left"></i> Vazgeç
                            </a>
                            <button type="submit" class="btn btn-success px-5">
                                <i class="bi bi-check-lg"></i> Branşı Kaydet
                            </button>
                        </div>
                    </form>
                </div>
            </div>
        </div>
    </div>
</div>

@section Scripts {
    @{
        await Html.RenderPartialAsync("_ValidationScriptsPartial");
    }
}

```

16.4 Beklenen Sonuç

- **Başarılı:** Branş eklenir, Index'e dönülür, sağ üst köşede yeşil Toastr "Branş başarıyla eklendi!" görünür.
- **Başarısız (boş ad):** Form geri döner, `BransAdi` altında kırmızı "Zorunlu alan" mesajı çıkar.

16.5 Notlar

- `ToUpper()` ile branş adı veritabanına büyük harfle yazılır; bu sayede aynı branşın farklı yazımları tek kayıt olur.
- `HttpPost, ActionName("Sil")` — POST endpoint'inin URL yolu hâlâ `/Brans/Sil` 'dir, ama metot adı `SilOnaylandi` olduğu için GET ve POST overload çakışması önlenir.

17. Doktor Yönetimi (CRUD)

17.1 Amaç

Doktorları branşlarıyla birlikte yönetir. Doktor ekleme/güncelleme formlarında branş dropdown'u zorunludur. Index sayfasında ad/branş bilgisiyle birlikte arama yapılabilir.

17.2 DoktorService

```
// HastaneMVC/Services/DoktorService.cs
using HastaneMVC.Data;
using HastaneMVC.Models;

namespace HastaneMVC.Services
{
    public class DoktorService : BaseService
    {
        public DoktorService(HastaneContext context) : base(context)
        {
        }

        public List<DoktorModel> indexgetir(string ara)
        {
            List<DoktorModel> doktorlistesi;

            if (ara != null)
            {
                doktorlistesi = _context.Doktorlar
                    .Where(d => d.Brans.BransAdi == ara.ToUpper() && d.IsDeleted == false)
                    .Select(d => new DoktorModel
                    {
                        Id = d.Id,
                        AdSoyad = d.AdSoyad,
                        BransId = d.BransId,
                        Brans = d.Brans
                    })
                    .OrderBy(i => i.AdSoyad)
                    .ToList();
            }
            else
            {
                doktorlistesi = _context.Doktorlar
                    .Where(d => d.IsDeleted == false)
                    .Select(d => new DoktorModel
                    {
                        Id = d.Id,
                        AdSoyad = d.AdSoyad,
                        BransId = d.BransId,
                        Brans = d.Brans
                    })
                    .ToList();
            }

            return doktorlistesi;
        }
    }
}
```

17.3 DoktorEkleVM

```
// HastaneMVC/ViewModels/DoktorEkleVM.cs
using HastaneMVC.DTOS;
using Microsoft.AspNetCore.Mvc.ModelBinding.Validation;

namespace HastaneMVC.ViewModels
{
    public class DoktorEkleVM
    {
        public DoktorModelDTO YeniDoktor { get; set; }

        [ValidateNever]
        public IEnumerable<BransModelDTO> BransListesi { get; set; }
    }
}
```

17.4 DoktorController

```
// HastaneMVC/Controllers/DoktorController.cs
using HastaneMVC.Data;
using HastaneMVC.DTOs;
using HastaneMVC.Models;
using HastaneMVC.Services;
using HastaneMVC.ViewModels;
using Microsoft.AspNetCore.Mvc;

namespace HastaneMVC.Controllers
{
    public class DoktorController : BaseController
    {
        private readonly DoktorService _doktorService;

        public DoktorController(HastaneContext context, DoktorService doktorService)
            : base(context)
        {
            _doktorService = doktorService;
        }

        public IActionResult Index(string ara)
        {
            var doktorlar = _doktorService.indexgetir(ara);

            var doktorDTolar = doktorlar.Select(d => new DoktorModelDTO
            {
                Id = d.Id,
                AdSoyad = d.AdSoyad,
                BransId = d.BransId,
                BransAdi = d.Brans?.BransAdi
            }).ToList();

            return View(doktorDTolar);
        }

        [HttpGet]
        public IActionResult Ekle()
        {
            var vm = new DoktorEkleVM
            {
                YeniDoktor = new DoktorModelDTO(),
                BransListesi = branslarigetir()
            };
            return View(vm);
        }

        [HttpPost]
        public IActionResult Ekle(DoktorEkleVM gelenKutu)
        {
            if (!ModelState.IsValid)
            {
                gelenKutu.BransListesi = branslarigetir();
                return View(gelenKutu);
            }

            var entity = new DoktorModel
            {

```

```

        AdSoyad = gelenKutu.YeniDoktor.AdSoyad,
        BransId = gelenKutu.YeniDoktor.BransId
    };

    _context.Doktorlar.Add(entity);
    _context.SaveChanges();
    TempData["success"] = "Doktor başarıyla eklendi!";
    return RedirectToAction("Index");
}

[HttpGet]
public IActionResult Guncelle(int id)
{
    var doktor = _context.Doktorlar.Find(id);
    if (doktor == null) return NotFound();

    var vm = new DoktorEkleVM
    {
        YeniDoktor = new DoktorModelDTO
        {
            Id = doktor.Id,
            AdSoyad = doktor.AdSoyad,
            BransId = doktor.BransId
        },
        BransListesi = branslarigetir()
    };
    return View(vm);
}

[HttpPost]
public IActionResult Guncelle(DoktorEkleVM gelenKutu)
{
    if (!ModelState.IsValid)
    {
        gelenKutu.BransListesi = branslarigetir();
        return View(gelenKutu);
    }

    var eskiDoktor = _context.Doktorlar.Find(gelenKutu.YeniDoktor.Id);
    eskiDoktor.AdSoyad = gelenKutu.YeniDoktor.AdSoyad;
    eskiDoktor.BransId = gelenKutu.YeniDoktor.BransId;

    _context.Doktorlar.Update(eskiDoktor);
    _context.SaveChanges();
    TempData["success"] = "Doktor güncellendi!";
    return RedirectToAction("Index");
}

[HttpGet]
public IActionResult Sil(int id)
{
    var doktor = _context.Doktorlar
        .Where(d => d.Id == id)
        .Select(d => new DoktorModelDTO
        {
            Id = d.Id,
            AdSoyad = d.AdSoyad,

```

```

        BransId = d.BransId,
        BransAdi = d.Brans.BransAdi
    })
    .FirstOrDefault();

    if (doktor == null) return NotFound();
    return View(doktor);
}

[HttpPost, ActionName("Sil")]
public IActionResult SilOnaylandi(int id)
{
    var doktor = _context.Doktorlar.Find(id);
    if (doktor != null)
    {
        doktor.IsDeleted = true;
        doktor.DeleteTime = DateTime.Now;
        _context.SaveChanges();
        TempData["success"] = "Doktor silindi!";
    }
    return RedirectToAction("Index");
}
}
}
}

```

17.5 DoktorModelDTO

```

// HastaneMVC/DTOs/DoktorModelDTO.cs
using System.ComponentModel.DataAnnotations;

namespace HastaneMVC.DTOs
{
    public class DoktorModelDTO
    {
        public int Id { get; set; }

        [Required(ErrorMessage = "Ad Soyad zorunludur")]
        [StringLength(150)]
        public string AdSoyad { get; set; }

        [Required(ErrorMessage = "Branş seçimi zorunludur")]
        public int BransId { get; set; }

        // Sadece listeleme için; [Required] yazılmaz
        public string BransAdi { get; set; }
    }
}

```

17.6 Ekle View (parça)

```
<!-- HastaneMVC/Views/Doktor/Ekle.cshtml -->
@using HastaneMVC.ViewModels
@model DoktorEkleVM

<form asp-action="Ekle" method="post">
    <div class="mb-3">
        <label asp-for="YeniDoktor.AdSoyad" class="form-label">Ad Soyad</label>
        <input asp-for="YeniDoktor.AdSoyad" class="form-control" />
        <span asp-validation-for="YeniDoktor.AdSoyad" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="YeniDoktor.BransId" class="form-label">Brans</label>
        <select asp-for="YeniDoktor.BransId"
            asp-items="@((new SelectList(Model.BransListesi, "Id", "BransAdi")))"
            class="form-select">
            <option value="">-- Brans Seçiniz --</option>
        </select>
        <span asp-validation-for="YeniDoktor.BransId" class="text-danger"></span>
    </div>

    <button type="submit" class="btn btn-success">Kaydet</button>
</form>

@section Scripts {
    @{
        await Html.RenderPartialAsync("_ValidationScriptsPartial");
    }
}
```

17.7 Notlar

- `DoktorModelDTO.BransAdi` yalnızca Index sayfasında listeleme için kullanılır; bu yüzden **[Required]** yazılmaz.
- Güncelle GET'inde entity manuel olarak DTO'ya çevrilir; entity doğrudan view'e dönmez.

18. Hasta Yönetimi (CRUD + Unique TC)

18.1 Amaç

Hastaların adını, soyadını ve TC kimlik numarasını yönetir. TC No 11 karakter olmak zorundadır ve veritabanında benzersizdir. Aynı TC ile iki kayıt oluşturulamaz.

18.2 HastaModelDTO

```
// HastaneMVC/DTOs/HastaModelDTO.cs
using System.ComponentModel.DataAnnotations;

namespace HastaneMVC.DTOs
{
    public class HastaModelDTO
    {
        public int Id { get; set; }

        [Required(ErrorMessage = "Ad Soyad zorunludur")]
        [StringLength(150)]
        public string AdSoyad { get; set; }

        [Required(ErrorMessage = "TC No zorunludur")]
        [StringLength(11, MinimumLength = 11, ErrorMessage = "TC No 11 karakter olmalıdır")]
        public string TcNo { get; set; }
    }
}
```

18.3 HastaController — Unique TC Kontrolü

```
// HastaneMVC/Controllers/HastaController.cs
[HttpPost]
public IActionResult Ekle(HastaModelDTO yeniHasta)
{
    if (!ModelState.IsValid)
    {
        return View(yeniHasta);
    }

    var ayniTcVarMi = _context.Hastalar
        .Any(h => h.TcNo == yeniHasta.TcNo && h.IsDeleted == false);

    if (ayniTcVarMi)
    {
        ModelState.AddModelError("TcNo", "Bu TC No zaten kayıtlı!");
        return View(yeniHasta);
    }

    var entity = new HastaModel
    {
        AdSoyad = yeniHasta.AdSoyad,
        TcNo = yeniHasta.TcNo
    };

    _context.Hastalar.Add(entity);
    _context.SaveChanges();
    TempData["success"] = "Hasta başarıyla kaydedildi!";
    return RedirectToAction("Index");
}

[HttpPost]
public IActionResult guncelle(HastaModelDTO guncelhasta)
{
    if (!ModelState.IsValid)
    {
        return View(guncelhasta);
    }

    var eskiHasta = _context.Hastalar.Find(guncelhasta.Id);
    eskiHasta.AdSoyad = guncelhasta.AdSoyad;
    eskiHasta.TcNo = guncelhasta.TcNo;

    _context.Hastalar.Update(eskiHasta);
    _context.SaveChanges();
    TempData["success"] = "Hasta bilgileri güncellendi!";
    return RedirectToAction("Index");
}

[HttpPost, ActionName("sil")]
public IActionResult silOnaylandi(int id)
{
    var hasta = _context.Hastalar.Find(id);
    if (hasta != null)
    {
        hasta.IsDeleted = true;
        hasta.DeleteTime = DateTime.Now;
        _context.SaveChanges();
    }
}
```

```
TempData["success"] = "Hasta silindi!";
}
return RedirectToAction("Index");
}
```

18.4 Veritabanı Seviyesinde Unique

```
// HastaneMVC/Data/HastaneContext.cs (parça)
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<HastaModel>()
        .HasIndex(h => h.TcNo)
        .IsUnique();
}
```

Migration komutu (Package Manager Console):

```
Add-Migration UniqueTC
Update-Database
```

Çıktı (SQL Server'da üretilen index):

```
CREATE UNIQUE INDEX [IX_Hastalar_TcNo] ON [Hastalar] ([TcNo]);
```

18.5 İki Katmanlı Koruma

Katman	Görev
Controller'da <code>Any(...)</code> kontrolü	Türkçe hata mesajı gösterir
Context'te <code>HasIndex().IsUnique()</code>	DB seviyesinde zorlar; controller atlansa bile korur

18.6 Yaygın Hatalar

18.6.1 — `eskiHasta.Id = guncelhasta.Id` satırı

Hatalı kod:

```
var eskiHasta = _context.Hastalar.Find(guncelhasta.Id);
eskiHasta.Id = guncelhasta.Id; // gereksiz
eskiHasta.AdSoyad = guncelhasta.AdSoyad;
```

Sebebi: `Find` zaten doğru kaydı ID'ye göre bulur; ardından Id'yi yeniden atamak gereksizdir ve EF Core'da bazı durumlarda hata fırlatır.

Düzeltilmiş kod: Bu satır kaldırılır.

18.7 Notlar

- `[StringLength(11, MinimumLength = 11)]` hem maksimum hem minimum uzunluğu zorlar.
- Soft delete sonrası aynı TC ile yeni kayıt eklemek istenirse, `Any` kontrolü `IsDeleted == false` filtresini içerdiği için izin verilir; DB unique index hâlâ silinmemiş kayıtlar arasında çakışmayı engeller.

19. Randevu Yönetimi (CRUD + AJAX)

19.1 Amaç

Hasta + doktor + tarih + triyaj notu kombinasyonuyla randevu oluşturur. Randevu sayfasındaki en kritik özellik **branş seçilince doktor listesinin AJAX ile dinamik filtrelenmesidir**.

19.2 Akış

```
sequenceDiagram
    actor Kullanıcı
    participant View as Randevu/Ekle.cshtml
    participant Controller as RandevuController
    participant DB as HastaneContext

    Kullanıcı->>View: Branş seç
    View->>Controller: GET /Randevu/DoktorlariGetir?bransId=11
    Controller->>DB: Where(BransId == 11)
    DB-->>Controller: Doktor listesi
    Controller-->>View: JSON
    View->>View: doktorSelect dropdown'unu doldur
    Kullanıcı->>View: Doktor + tarih + not gir, Kaydet
    View->>Controller: POST /Randevu/Ekle
    Controller->>DB: Add(RandevuModel)
    DB-->>Controller: OK
    Controller-->>View: Redirect /Randevu/Index (TempData["success"])
```

19.3 RandevuService

```
// HastaneMVC/Services/RandevuService.cs
using HastaneMVC.Data;
using HastaneMVC.DTOs;
using Microsoft.EntityFrameworkCore;

namespace HastaneMVC.Services
{
    public class RandevuService : BaseService
    {
        public RandevuService(HastaneContext context) : base(context)
        {
        }

        public List<RandevuModelDTO> tumunuGetir()
        {
            return _context.Randevular
                .Include(r => r.Doktor)
                .ThenInclude(d => d.Brans)
                .Include(r => r.Hasta)
                .Where(r => r.IsDeleted == false)
                .Select(r => new RandevuModelDTO
                {
                    Id = r.Id,
                    HastaId = r.HastaId,
                    HastaAdSoyad = r.Hasta.AdSoyad,
                    DoktorId = r.DoktorId,
                    DoktorAdSoyad = r.Doktor.AdSoyad,
                    BransId = r.Doktor.BransId,
                    BransAdi = r.Doktor.Brans.BransAdi,
                    TarihSaat = r.TarihSaat,
                    DoktorNotu = r.DoktorNotu
                })
                .ToList();
        }

        public RandevuModelDTO guncellenecekOlanGetir(int id)
        {
            var randevu = _context.Randevular
                .Where(i => i.Id == id)
                .Select(r => new RandevuModelDTO
                {
                    Id = r.Id,
                    HastaId = r.HastaId,
                    DoktorId = r.DoktorId,
                    TarihSaat = r.TarihSaat,
                    DoktorNotu = r.DoktorNotu,
                    BransId = r.Doktor.BransId
                })
                .FirstOrDefault();

            if (randevu == null) return new RandevuModelDTO();
            return randevu;
        }

        public void SilPost(int id)
        {
            var randevu = _context.Randevular.Find(id);
        }
    }
}
```

```
        if (randevu != null)
        {
            randevu.IsDeleted = true;
            randevu.DeleteTime = DateTime.Now;
            _context.SaveChanges();
        }
    }
}
```

19.4 RandevuController — Tam Kod

```
// HastaneMVC/Controllers/RandevuController.cs
using HastaneMVC.Data;
using HastaneMVC.DTOs;
using HastaneMVC.Models;
using HastaneMVC.Services;
using HastaneMVC.ViewModels;
using Microsoft.AspNetCore.Mvc;

namespace HastaneMVC.Controllers
{
    public class RandevuController : BaseController
    {
        private readonly RandevuService _randevuService;

        public RandevuController(HastaneContext context, RandevuService randevuService)
            : base(context)
        {
            _randevuService = randevuService;
        }

        public IActionResult Index()
        {
            var randevular = _randevuService.tumunuGetir();
            return View(randevular);
        }

        [HttpGet]
        public IActionResult Ekle()
        {
            var vm = new RandevuEkleVM
            {
                YeniRandevu = new RandevuModelDTO(),
                DoktorListesi = doktorgetir(),
                HastaListesi = hastalarigetir(),
                BransListesi = branslarigetir()
            };
            return View(vm);
        }

        [HttpPost]
        public IActionResult Ekle(RandevuEkleVM model)
        {
            if (!ModelState.IsValid)
            {
                model.DoktorListesi = doktorgetir();
                model.HastaListesi = hastalarigetir();
                model.BransListesi = branslarigetir();
                return View(model);
            }

            if (string.IsNullOrEmpty(model.YeniRandevu.DoktorNotu))
            {
                model.YeniRandevu.DoktorNotu = "Henüz not girilmedi.";
            }

            var entity = new RandevuModel
            {

```

```

        HastaId = model.YeniRandevu.HastaId,
        DoktorId = model.YeniRandevu.DoktorId,
        TarihSaat = model.YeniRandevu.TarihSaat,
        DoktorNotu = model.YeniRandevu.DoktorNotu
    };

    _context.Randevular.Add(entity);
    _context.SaveChanges();
    TempData["success"] = "Randevu oluşturuldu!";
    return RedirectToAction("Index");
}

[HttpGet]
public IActionResult Guncelle(int id)
{
    var viewModel = new RandevuEkleVM
    {
        YeniRandevu = _randevuService.guncellenecekOlanGetir(id),
        DoktorListesi = doktorgetir(),
        HastaListesi = hastalarigetir(),
        BransListesi = branslarigetir()
    };
    return View(viewModel);
}

[HttpPost]
public IActionResult Guncelle(RandevuEkleVM model)
{
    if (!ModelState.IsValid)
    {
        model.DoktorListesi = doktorgetir();
        model.HastaListesi = hastalarigetir();
        model.BransListesi = branslarigetir();
        return View(model);
    }

    var eskiRandevu = _context.Randevular.Find(model.YeniRandevu.Id);
    eskiRandevu.HastaId = model.YeniRandevu.HastaId;
    eskiRandevu.DoktorId = model.YeniRandevu.DoktorId;
    eskiRandevu.TarihSaat = model.YeniRandevu.TarihSaat;
    eskiRandevu.DoktorNotu = model.YeniRandevu.DoktorNotu;

    _context.Randevular.Update(eskiRandevu);
    _context.SaveChanges();
    TempData["success"] = "Randevu güncellendi!";
    return RedirectToAction("Index");
}

[HttpGet]
public IActionResult Sil(int id)
{
    var randevu = _randevuService.guncellenecekOlanGetir(id);
    return View(randevu);
}

[HttpPost, ActionName("Sil")]
public IActionResult SilOnaylandi(int id)

```

```

    {
        _randevuService.SilPost(id);
        TempData["success"] = "Randevu iptal edildi!";
        return RedirectToAction("Index");
    }

    [HttpGet]
    public JsonResult DoktorlariGetir(int bransId)
    {
        var doktorlar = _context.Doktorlar
            .Where(d => d.BransId == bransId && d.IsDeleted == false)
            .Select(d => new
            {
                id = d.Id,
                adSoyad = d.AdSoyad
            })
            .ToList();

        return Json(doktorlar);
    }

    private IEnumerable<DoktorModelDTO> doktorgetir()
    {
        return _context.Doktorlar
            .Where(d => d.IsDeleted == false)
            .Select(d => new DoktorModelDTO
            {
                Id = d.Id,
                AdSoyad = d.AdSoyad,
                BransId = d.BransId
            })
            .ToList();
    }

    private IEnumerable<HastaModelDTO> hastalarigetir()
    {
        return _context.Hastalar
            .Where(h => h.IsDeleted == false)
            .Select(h => new HastaModelDTO
            {
                Id = h.Id,
                AdSoyad = h.AdSoyad,
                TcNo = h.TcNo
            })
            .ToList();
    }
}

```

19.5 RandevuModelDTO

```
// HastaneMVC/DTOs/RandevuModelDTO.cs
using System.ComponentModel.DataAnnotations;

namespace HastaneMVC.DTOs
{
    public class RandevuModelDTO
    {
        public int Id { get; set; }

        [Required(ErrorMessage = "Hasta seçimi zorunludur")]
        public int HastaId { get; set; }

        [Required(ErrorMessage = "Doktor seçimi zorunludur")]
        public int DoktorId { get; set; }

        [Required(ErrorMessage = "Tarih ve saat zorunludur")]
        public DateTime TarihSaat { get; set; }

        public string DoktorNotu { get; set; }

        public int BransId { get; set; }

        // Sadece listeleme için
        public string HastaAdSoyad { get; set; }
        public string DoktorAdSoyad { get; set; }
        public string BransAdi { get; set; }
    }
}
```

19.6 Ekle View — Tam Kod (kritik kısımlar)

```

<!-- HastaneMVC/Views/Randevu/Ekle.cshtml -->
@using HastaneMVC.ViewModels
@model RandevuEkleVM

@{
    ViewData["Title"] = "Yeni Randevu";
}

<div class="container mt-5">
    <div class="card shadow border-0">
        <div class="card-header bg-primary text-white py-3">
            <h5 class="mb-0"><i class="bi bi-calendar-plus"></i> Yeni Randevu Kayıt Formu</h5>
        </div>
        <div class="card-body p-4">
            <form asp-action="Ekle" method="post">
                <div class="mb-3">
                    <label asp-for="YeniRandevu.HastaId" class="form-label">Randevu Alacak Hasta</label>
                    <select asp-for="YeniRandevu.HastaId"
                        asp-items="@((new SelectList(Model.HastalListesi, "Id", "AdSoyad")))"
                        class="form-select">
                        <option value="">-- Lütfen Hasta Seçiniz --</option>
                    </select>
                    <span asp-validation-for="YeniRandevu.HastaId" class="text-danger"></span>
                </div>

                <div class="mb-3">
                    <label class="form-label">Branş</label>
                    <select id="bransSelect"
                        asp-items="@((new SelectList(Model.BransListesi, "Id", "BransAdi")))"
                        class="form-select">
                        <option value="">-- Önce Branş Seçiniz --</option>
                    </select>
                </div>

                <div class="mb-3">
                    <label asp-for="YeniRandevu.DoktorId" class="form-label">Gidilecek Doktor</label>
                    <select asp-for="YeniRandevu.DoktorId" id="doktorSelect" class="form-select">
                        <option value="">-- Lütfen Önce Branş Seçiniz --</option>
                    </select>
                    <small class="text-muted">Doktor listesi seçilen branşa göre otomatik yüklenecektir.
                <span asp-validation-for="YeniRandevu.DoktorId" class="text-danger"></span>
                </div>

                <div class="mb-3">
                    <label asp-for="YeniRandevu.TarihSaat" class="form-label">Randevu Tarihi ve Saati</label>
                    <input asp-for="YeniRandevu.TarihSaat" type="datetime-local"
                        class="form-control"
                        value="@DateTime.Now.ToString("yyyy-MM-ddTHH:mm")" />
                    <span asp-validation-for="YeniRandevu.TarihSaat" class="text-danger"></span>
                </div>

                <div class="mb-3">
                    <label asp-for="YeniRandevu.DoktorNotu" class="form-label">Triyaj Notu</label>
                    <textarea asp-for="YeniRandevu.DoktorNotu" rows="4" class="form-control"
                        placeholder="Triyaj notu giriniz..."></textarea>
                </div>
            </form>
        </div>
    </div>

```

```

        <div class="d-flex justify-content-end gap-2">
            <a asp-action="Index" class="btn btn-outline-secondary">
                <i class="bi bi-arrow-left"></i> Listeye Dön
            </a>
            <button type="submit" class="btn btn-primary">
                <i class="bi bi-calendar-check"></i> Randevuyu Onayla
            </button>
        </div>
    </form>
</div>
</div>
</div>

@section Scripts {
    @{
        await Html.RenderPartialAsync("_ValidationScriptsPartial");
    }

    <script>
        $(document).ready(function () {
            $('#bransSelect').change(BransDegisti);
        });

        function BransDegisti() {
            var secilenBransId = $('#bransSelect').val();
            if (secilenBransId != "") {
                AjaxIleDoktorlariGetir(secilenBransId);
            } else {
                DoktorKutusunuKilitle();
            }
        }

        function AjaxIleDoktorlariGetir(bransId) {
            $.ajax({
                url: '/Randevu/DoktorlariGetir',
                type: 'GET',
                data: { bransId: bransId },
                success: DoktorKutusunuDoldur,
                error: SunucuHatasi
            });
        }

        function DoktorKutusunuDoldur(gelenVeri) {
            var doktorSelect = $('#doktorSelect');
            doktorSelect.empty();
            doktorSelect.append('<option value="">-- Lütfen Doktor Seçiniz --</option>');
            for (var i = 0; i < gelenVeri.length; i++) {
                var doktor = gelenVeri[i];
                doktorSelect.append('<option value="' + doktor.id + '">' + doktor.adSoyad + '</option>');
            }
        }

        function DoktorKutusunuKilitle() {
            var doktorSelect = $('#doktorSelect');
            doktorSelect.empty();
            doktorSelect.append('<option value="">-- Lütfen Önce Branş Seçiniz --</option>');
        }
    </script>
}

```

```
function SunucuHatasi() {  
    alert("Sunucu hatası oluştu!");  
}  
</script>  
}
```

19.7 Guncelle View — AJAX'ın Farkı

Güncelleme sayfası açıldığında mevcut branş otomatik seçili gelmelidir. Bunun için bir gizli input ile branş ID'si taşınır:

```

<!-- HastaneMVC/Views/Randevu/Guncelle.cshtml (script bölümü) -->
@section Scripts {
    @{
        await Html.RenderPartialAsync("_ValidationScriptsPartial");
    }

    <script>
        $(document).ready(function () {
            $('#bransSelect').change(BransDegisti);
            var mevcutBransId = $('#gizliBransId').val();
            $('#bransSelect').val(mevcutBransId);
            AjaxIleDoktorlariGetir(mevcutBransId);
        });

        function BransDegisti() {
            var secilenBransId = $('#bransSelect').val();
            if (secilenBransId != "") {
                AjaxIleDoktorlariGetir(secilenBransId);
            } else {
                DoktorKutusunuKilitle();
            }
        }

        function AjaxIleDoktorlariGetir(bransId) {
            $.ajax({
                url: '/Randevu/DoktorlariGetir',
                type: 'GET',
                data: { bransId: bransId },
                success: DoktorKutusunuDoldur,
                error: SunucuHatasi
            });
        }

        function DoktorKutusunuDoldur(gelenVeri) {
            var doktorSelect = $('#doktorSelect');
            doktorSelect.empty();
            doktorSelect.append('<option value="">-- Lütfen Doktor Seçiniz --</option>');
            for (var i = 0; i < gelenVeri.length; i++) {
                var doktor = gelenVeri[i];
                doktorSelect.append('<option value="' + doktor.id + '">' + doktor.adSoyad + '</option>');
            }
        }

        function SunucuHatasi() {
            alert("Sunucu hatası oluştu!");
        }
    </script>
}

```

Gerekli gizli input:

```
<input type="hidden" id="gizliBransId" value="@Model.YeniRandevu.BransId" />
```

19.8 Beklenen Sonuç

- **Branş seçilince:** O branşa bağlı doktorlar AJAX ile gelir, doktor dropdown'u dolar.
- **Branş seçilmediyse:** Doktor dropdown'u "-- Lütfen Önce Branş Seçiniz --" mesajıyla kilitli kalır.
- **Triyaj notu boşsa:** Otomatik olarak "Henüz not girilmedi." atanır.
- **Tarih boşsa:** ModelState `false` döner; validation mesajı gösterilir.

19.9 Notlar

- `RandevuService.guncellenecekOlanGetir` içinde `BransId = r.Doktor.BransId` ile branş ID'si çıkarılır; bu olmadan güncelleme sayfasında mevcut branş seçili gelmez.
- `DoktorNotu` opsiyoneldir; controller boşsa default değer atar.
- `type="datetime-local"` tarayıcının yerleşik tarih/saat seçicisini kullanır; ekstra kütüphane gerekmez.

20. Tahlil Türü Yönetimi (CRUD)

20.1 Amaç

Tahlil türlerini (Hemogram, Kan Şekeri, vb.) yönetir. Randevu sırasında istenecek tahlillerin türünü tanımlar.

20.2 TahlilTurModelDTO

```
// HastaneMVC/DTOs/TahlilTurModelDTO.cs
using System.ComponentModel.DataAnnotations;

namespace HastaneMVC.DTOs
{
    public class TahlilTurModelDTO
    {
        public int Id { get; set; }

        [Required(ErrorMessage = "Zorunlu alan")]
        [StringLength(150, ErrorMessage = "En fazla 150 karakter olabilir")]
        public string Ad { get; set; }
    }
}
```

20.3 TahlilTurController — Ekle Action

```
// HastaneMVC/Controllers/TahlilTurController.cs
[HttpPost]
public IActionResult Ekle(TahlilTurModelDTO modelDTO)
{
    if (ModelState.IsValid)
    {
        var entity = new TahlilTurModel
        {
            Ad = modelDTO.Ad
        };

        _context.TahlilTurleri.Add(entity);
        _context.SaveChanges();
        TempData["success"] = "Tahlil türü eklendi!";
        return RedirectToAction("Index");
    }
    return View(modelDTO);
}
```

20.4 Yaygın Hata

20.4.1 — Yeni kayıta Id'yi manuel atamak

Hatalı kod:

```
var entity = new TahlilTurModel
{
    Ad = modelDTO.Ad,
    Id = modelDTO.Id // YANLIŞ – yeni kayıta Id otomatik atanır
};
```

Sebebi: Primary key `int` tipi `[Key]` olarak işaretliyse EF Core onu otomatik artırır. Manuel atama 0 değeriyle yapılırsa sorun çıkarmaz ama gereksizdir; başka bir değer atanırsa duplicate key hatası alınır.

Düzeltilmiş kod: Yeni entity oluştururken `Id` atanmaz.

21. Tahlil Sonuç Yönetimi (CRUD)

21.1 Amaç

Bir randevuda istenen tahlilleri ve sonuçlarını yönetir. Bir randevu birden çok tahlile bağlanabilir (çoka-çok ilişki); `RandevuTahlilModel` bu ilişkinin bağlantı tablosudur ve `Sonuc` alanı ile zenginleştirilmiştir.

21.2 RandevuTahlilEkleVM

```
// HastaneMVC/ViewModels/RandevuTahlilEkleVM.cs
using HastaneMVC.DTOS;
using Microsoft.AspNetCore.Mvc.ModelBinding.Validation;

namespace HastaneMVC.ViewModels
{
    public class RandevuTahlilEkleVM
    {
        public RandevuTahlilModelDTO YeniTahlil { get; set; }

        [ValidateNever]
        public IEnumerable<RandevuModelDTO> RandevuListesi { get; set; }

        [ValidateNever]
        public IEnumerable<TahlilTurModelDTO> TahlilTurListesi { get; set; }
    }
}
```

21.3 RandevuTahlilController — Sil Action

```
// HastaneMVC/Controllers/RandevuTahlilController.cs
[HttpPost, ActionName("Sil")]
public IActionResult SilOnaylandi(int id)
{
    var tahlil = _context.RandevuTahlilleri.Find(id);
    if (tahlil != null)
    {
        tahlil.IsDeleted = true;
        tahlil.DeleteTime = DateTime.Now;
        _context.SaveChanges();
        TempData["success"] = "Tahlil isteği iptal edildi!";
    }
    return RedirectToAction("Index");
}
```

21.4 Yaygın Hata

21.4.1 — DeleteTime atamasını if bloğunun dışına yazmak

Hatalı kod:

```
var tahlil = _context.RandevuTahlilleri.Find(id);
tahlil.DeleteTime = DateTime.Now; // tahlil null olabilir!
if (tahlil != null)
{
    tahlil.IsDeleted = true;
    _context.SaveChanges();
}
```

Hata mesajı:

```
System.NullReferenceException: Object reference not set to an instance of an object.
```

Sebebi: `Find` kayıt bulamazsa `null` döner; `null.DeleteTime` çağrısı `NullReferenceException` fırlatır.

Düzeltilmiş kod: Tüm atamalar `if (tahlil != null)` bloğu içine alınır.

EKLER

Ek A — Connection String ve Migration

A.1 appsettings.json

```
// HastaneMVC/appsettings.json
{
  "ConnectionStrings": {
    "HastaneBaglantisi": "YOUR_CONNECTION_STRING"
  }
}
```

Tipik LocalDB değeri (gerçek projede placeholder ile değiştirilir):

```
Server=(localdb)\MSSQLLocalDB;Database=HastaneMVCDData;Trusted_Connection=True;TrustServerCertificate=True
```

A.2 Program.cs'te Okuma

```
// HastaneMVC/Program.cs (parça)
builder.Services.AddDbContext<HastaneContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("HastaneBaglantisi")));
```

A.3 Migration Komutları

Package Manager Console'da çalıştırılır:

```
Add-Migration MigrationAdi
Update-Database
```

Veya .NET CLI:

```
dotnet ef migrations add MigrationAdi
dotnet ef database update
```

Geri alma:

```
Update-Database OncekiMigrationAdi
Remove-Migration
```

Ek B — Git ile Kayıt ve GitHub'a Yükleme

B.1 Visual Studio Üzerinden

1. **Git → Create Git Repository** → GitHub seçilir, repo adı ve görünülük belirlenir.
2. **Create and Push** butonu hem yerel Git deposu oluşturur hem GitHub'a yükler.

B.2 Sonraki Commit'ler

1. **View → Git Changes** panelini aç.
2. Mesaj alanına commit açıklaması yaz.
3. **Commit All** ile yerel kaydet, ardından ↑ (push) ile GitHub'a gönder.
4. Veya **Commit All and Push** ile ikisini tek adımda yap.

B.3 Terminal Üzerinden (Git Bash)

```
cd "C:/proje/yolu/HastaneMVC"
git add .
git commit -m "Toastr bildirimleri eklendi"
git push origin master
```

Not: PowerShell'de **git** komutu tanınmıyorsa Git for Windows kurulu değildir veya PATH'e eklenmemiştir. Git Bash ile aynı komutlar çalışır.

B.4 Solution Explorer'daki Git Simgeleri

Simge	Anlamı
Mavi kilit	Commit edilmiş, değişiklik yok
Sarı kalem	Değiştirilmiş, commit bekliyor
Yeşil artı	Yeni eklenmiş, henüz commit'lenmemiş
Mor soru işareti	Git'e eklenmemiş dosya